arXiv:2404.19737v1 [cs.CL] 30 Apr 2024

Better & Faster Large Language Models via Multi-token Prediction

Fabian Gloeckle [*1 2] **Badr Youbi Idrissi** [*1 3] **Baptiste Rozière** [1] **David Lopez-Paz** [+1] **Gabriel Synnaeve** [+1]

Abstract

Large language models such as GPT and Llama are trained with a next-token prediction loss. In this work, we suggest that training language models to predict *multiple* future tokens at once results in higher sample efficiency. More specifically, at each position in the training corpus, we ask the model to predict the following n tokens using n independent output heads, operating on top of a shared model trunk. Considering multi-token prediction as an auxiliary training task, we measure improved downstream capabilities with no overhead in training time for both code and natural language models. The method is increasingly useful for larger model sizes, and keeps its appeal when training for multiple epochs. Gains are especially pronounced on *generative* benchmarks like coding, where our models consistently outperform strong baselines by several percentage points. Our 13B parameter models solves 12 % more problems on HumanEval and 17 % more on MBPP than comparable next-token models. Experiments on small algorithmic tasks demonstrate that multi-token prediction is favorable for the development of induction heads and algorithmic reasoning capabilities. As an additional benefit, models trained with 4-token prediction are up to 3× faster at inference, even with large batch sizes.

1. Introduction

Humanity has condensed its most ingenious undertakings, surprising findings and beautiful productions into text. Large Language Models (LLMs) trained on all of these corpora are able to extract impressive amounts of world knowledge, as well as basic reasoning capabilities by implementing a simple—yet powerful—unsupervised learning task: next-token prediction. Despite the recent wave of impressive achievements (OpenAI, 2023), next-token prediction remains an inefficient way of acquiring language, world knowledge and reasoning capabilities. More precisely, teacher forcing with next-token prediction latches on local patterns and overlooks "hard" decisions. Consequently, it remains a fact that state-of-the-art next-token predictors call for orders of magnitude more data than human children to arrive at the same level of fluency (Frank, 2023).

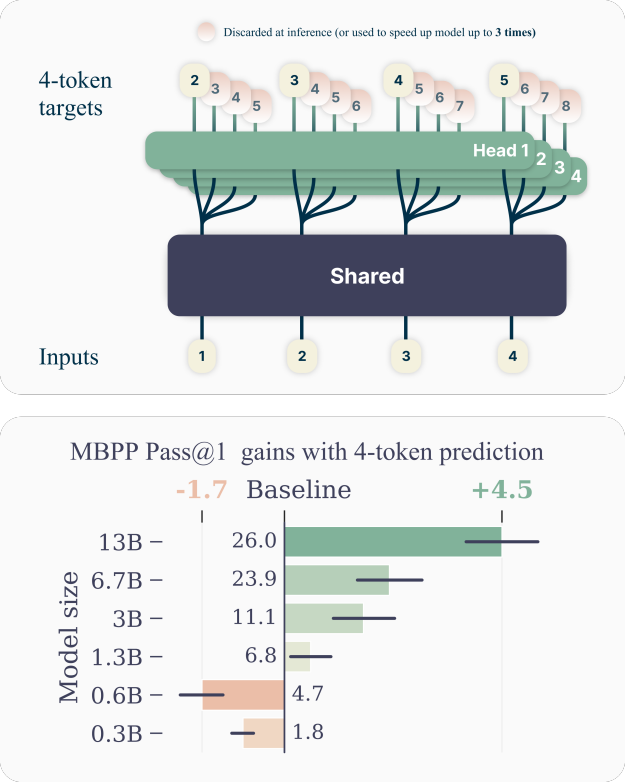


Figure 1: **Overview of multi-token prediction.** (Top) During training, the model predicts 4 future tokens at once, by means of a shared trunk and 4 dedicated output heads. During inference, we employ only the next-token output head. Optionally, the other three heads may be used to speed-up inference time. (Bottom) Multi-token prediction improves pass@1 on the MBPP code task, significantly so as model size increases. Error bars are confidence intervals of 90% computed with bootstrapping over dataset samples.

[*]Equal contribution [+]Last authors [1]FAIR at Meta [2]CERMICS Ecole des Ponts ParisTech [3]LISN Université Paris-Saclay. Correspondence to: Fabian Gloeckle <fgloeckle@meta.com>, Badr Youbi Idrissi <byoubi@meta.com>.

In this study, we argue that training LLMs to *predict multiple tokens* at once will drive these models toward better sample efficiency. As anticipated in Figure 1, multi-token prediction instructs the LLM to predict the n future tokens from each position in the training corpora, all at once and in parallel (Qi et al., 2020).

Contributions While multi-token prediction has been studied in previous literature (Qi et al., 2020), the present work offers the following contributions:

1. We propose a simple multi-token prediction architecture with no train time or memory overhead (Section 2).
2. We provide experimental evidence that this training paradigm is beneficial at scale, with models up to 13B parameters solving around 15% more code problems on average (Section 3).
3. Multi-token prediction enables self-speculative decoding, making models up to 3 times faster at inference time across a wide range of batch-sizes (Section 3.2).

While cost-free and simple, multi-token prediction is an effective modification to train stronger and faster transformer models. We hope that our work spurs interest in novel auxiliary losses for LLMs well beyond next-token prediction, as to improve the performance, coherence, and reasoning abilities of these fascinating models.

2. Method

Standard language modeling learns about a large text corpus $x_1, \dots x_T$ by implementing a next-token prediction task. Formally, the learning objective is to minimize the cross-entropy loss

$$L_1 = -\sum_t \log P_\theta(x_{t+1} \mid x_{t:1}), \tag{1}$$

where P_θ is our large language model under training, as to maximize the probability of x_{t+1} as the next future token, given the history of past tokens $x_{t:1} = x_t, \dots, x_1$.

In this work, we generalize the above by implementing a multi-token prediction task, where at each position of the training corpus, the model is instructed to predict n future tokens at once. This translates into the cross-entropy loss

$$L_n = -\sum_t \log P_\theta(x_{t+n:t+1} \mid x_{t:1}). \tag{2}$$

To make matters tractable, we assume that our large language model P_θ employs a shared trunk to produce a latent representation $z_{t:1}$ of the observed context $x_{t:1}$, then fed into n independent heads to predict in parallel each of the n future tokens (see Figure 1). This leads to the following factorization of the multi-token prediction cross-entropy loss:

$$\begin{aligned} L_n &= -\sum_t \log P_\theta(x_{t+n:t+1} \mid z_{t:1}) \cdot P_\theta(z_{t:1} \mid x_{t:1}) \\ &= -\sum_t \sum_{i=1}^{n} \log P_\theta(x_{t+i} \mid z_{t:1}) \cdot P_\theta(z_{t:1} \mid x_{t:1}). \end{aligned}$$

In practice, our architecture consists of a shared transformer trunk f_s producing the hidden representation $z_{t:1}$ from the observed context $x_{t:1}$, n independent output heads implemented in terms of transformer layers f_{h_i}, and a shared unembedding matrix f_u. Therefore, to predict n future tokens, we compute:

$$P_\theta(x_{t+i} \mid x_{t:1}) = \text{softmax}(f_u(f_{h_i}(f_s(x_{t:1})))),$$

for $i = 1, \dots n$, where, in particular, $P_\theta(x_{t+1} \mid x_{t:1})$ is our next-token prediction head. See Appendix B for other variations of multi-token prediction architectures.

Memory-efficient implementation One big challenge in training multi-token predictors is reducing their GPU memory utilization. To see why this is the case, recall that in current LLMs the vocabulary size V is much larger than the dimension d of the latent representation—therefore, logit vectors become the GPU memory usage bottleneck. Naive implementations of multi-token predictors that materialize all logits and their gradients, both of shape (n, V), severely limit the allowable batch-size and average GPU memory utilization. Because of these reasons, in our architecture we propose to carefully adapt the sequence of forward and backward operations, as illustrated in Figure 2. In particular, after the forward pass through the shared trunk f_s, we sequentially compute the forward *and* backward pass of each independent output head f_i, accumulating gradients at the trunk. While this creates logits (and their gradients) for the output head f_i, these are freed before continuing to the next output head f_{i+1}, requiring the long-term storage only of the d-dimensional trunk gradient $\partial L_n / \partial f_s$. In sum, we have reduced the peak GPU memory utilization from $O(nV + d)$ to $O(V + d)$, at no expense in runtime (Table S5).

Inference During inference time, the most basic use of the proposed architecture is vanilla next-token autoregressive prediction using the next-token prediction head $P_\theta(x_{t+1} \mid x_{t:1})$, while discarding all others. However, the additional output heads can be leveraged to speed up decoding from the next-token prediction head with *self-speculative decoding* methods such as blockwise parallel decoding (Stern et al., 2018)—a variant of speculative decoding (Leviathan et al., 2023) without the need for an additional draft model—and speculative decoding with Medusa-like tree attention (Cai et al., 2024).

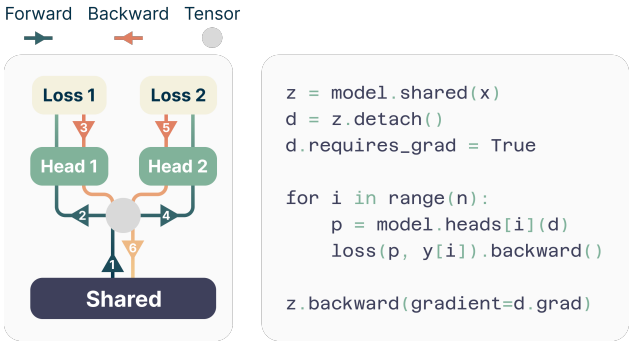


Figure 2: **Order of the forward/backward in an n-token prediction model with $n = 2$ heads.** By performing the forward/backward on the heads in sequential order, we avoid materializing all unembedding layer gradients in memory simultaneously and reduce peak GPU memory usage.

3. Experiments on real data

We demonstrate the efficacy of multi-token prediction losses by seven large-scale experiments. Section 3.1 shows how multi-token prediction is increasingly useful when growing the model size. Section 3.2 shows how the additional prediction heads can speed up inference by a factor of 3× using speculative decoding. Section 3.3 demonstrates how multi-token prediction promotes learning longer-term patterns, a fact most apparent in the extreme case of byte-level tokenization. Section 3.4 shows that 4-token predictor leads to strong gains with a tokenizer of size 32k. Section 3.5 illustrates that the benefits of multi-token prediction remain for training runs with multiple epochs. Section 3.6 showcases the rich representations promoted by pretraining with multi-token prediction losses by finetuning on the CodeContests dataset (Li et al., 2022). Section 3.7 shows that the benefits of multi-token prediction carry to natural language models, improving *generative* evaluations such as summarization, while not regressing significantly on standard benchmarks based on multiple choice questions and negative log-likelihoods.

To allow fair comparisons between next-token predictors and n-token predictors, the experiments that follow always compare models with an equal amount of parameters. That is, when we add $n - 1$ layers in future prediction heads, we remove $n - 1$ layers from the shared model trunk. Please refer to Table S14 for the model architectures and to Table S13 for an overview of the hyperparameters we use in our experiments.

3.1. Benefits scale with model size

To study this phenomenon, we train models of six sizes in the range 300M to 13B parameters from scratch on at least 91B tokens of code. The evaluation results in Figure 3 for MBPP (Austin et al., 2021) and HumanEval (Chen et al., 2021) show that it is possible, with the exact same computational budget, to squeeze much more performance out of large language models given a fixed dataset using multi-token prediction.

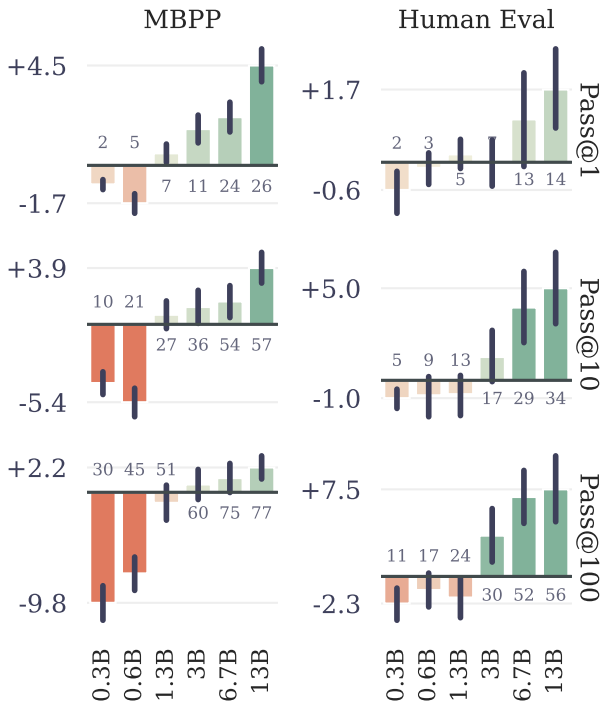


Figure 3: **Results of n-token prediction models on MBPP by model size.** We train models of six sizes in the range or 300M to 13B total parameters on code, and evaluate pass@1,10,100 on the MBPP (Austin et al., 2021) and HumanEval (Chen et al., 2021) benchmark with 1000 samples. Multi-token prediction models are worse than the baseline for small model sizes, but outperform the baseline at scale. Error bars are confidence intervals of 90% computed with bootstrapping over dataset samples.

We believe this *usefulness only at scale* to be a likely reason why multi-token prediction has so far been largely overlooked as a promising training loss for large language model training.

3.2. Faster inference

We implement greedy *self-speculative decoding* (Stern et al., 2018) with heterogeneous batch sizes using xFormers (Lefaudeux et al., 2022) and measure decoding speeds of our best 4-token prediction model with 7B parameters on completing prompts taken from a test dataset of code and natural language (Table S2) not seen during training. We observe a speedup of $\mathbf{3.0}\times$ on code with an average of 2.5 accepted tokens out of 3 suggestions on code, and of

Table 1: **Multi-token prediction improves performance and unlocks efficient byte level training.** We compare models with 7B parameters trained from scratch on 200B and on 314B bytes of code on the MBPP (Austin et al., 2021), HumanEval (Chen et al., 2021) and APPS (Hendrycks et al., 2021) benchmarks. Multi-token prediction largely outperforms next token prediction on these settings. All numbers were calculated using the estimator from Chen et al. (2021) based on 200 samples per problem. The temperatures were chosen optimally (based on test scores; i.e. these are oracle temperatures) for each model, dataset and pass@k and are reported in Table S12.

Training data	Vocabulary	n	MBPP			HumanEval			APPS/Intro		
			@1	@10	@100	@1	@10	@100	@1	@10	@100
313B bytes (0.5 epochs)	bytes	1	19.3	42.4	64.7	18.1	28.2	47.8	0.1	0.5	2.4
		8	**32.3**	**50.0**	**69.6**	**21.8**	**34.1**	**57.9**	**1.2**	**5.7**	**14.0**
		16	28.6	47.1	68.0	20.4	32.7	54.3	1.0	5.0	12.9
		32	23.0	40.7	60.3	17.2	30.2	49.7	0.6	2.8	8.8
200B tokens (0.8 epochs)	32k tokens	1	30.0	53.8	73.7	22.8	36.4	62.0	2.8	7.8	17.4
		2	30.3	55.1	76.2	22.2	38.5	62.6	2.1	9.0	21.7
		4	**33.8**	**55.9**	**76.9**	**24.0**	**40.1**	**66.1**	1.6	7.1	19.9
		6	31.9	53.9	73.1	20.6	38.4	63.9	**3.5**	**10.8**	**22.7**
		8	30.7	52.2	73.4	20.0	36.6	59.6	3.5	10.4	22.1
1T tokens (4 epochs)	32k tokens	1	40.7	65.4	83.4	31.7	57.6	83.0	**5.4**	**17.8**	**34.1**
		4	**43.1**	65.9	83.7	31.6	57.3	**86.2**	4.3	15.6	33.7

$2.7\times$ on text. On an 8-byte prediction model, the inference speedup is $6.4\times$ (Table S3). Pretraining with multi-token prediction allows the additional heads to be much more accurate than a simple finetuning of a next-token prediction model, thus allowing our models to unlock self-speculative decoding's full potential.

3.3. Learning global patterns with multi-byte prediction

To show that the next-token prediction task latches to local patterns, we went to the extreme case of byte-level tokenization by training a 7B parameter byte-level transformer on 314B bytes, which is equivalent to around 116B tokens. The 8-byte prediction model achieves astounding improvements compared to next-byte prediction, solving 67% more problems on MBPP pass@1 and 20% more problems on HumanEval pass@1.

Multi-byte prediction is therefore a very promising avenue to unlock efficient training of byte-level models. Self-speculative decoding can achieve speedups of 6 times for the 8-byte prediction model, which would allow to fully compensate the cost of longer byte-level sequences at inference time and even be faster than a next-token prediction model by nearly two times. The 8-byte prediction model is a strong byte-based model, approaching the performance of token-based models despite having been trained on $1.7\times$ less data.

3.4. Searching for the optimal n

To better understand the effect of the number of predicted tokens, we did comprehensive ablations on models of scale 7B trained on 200B tokens of code. We try $n = 1, 2, 4, 6$ and 8 in this setting. Results in table 1 show that training with 4-future tokens outperforms all the other models consistently throughout HumanEval and MBPP for pass at 1, 10 and 100 metrics: +3.8%, +2.1% and +3.2% for MBPP and +1.2%, +3.7% and +4.1% for HumanEval. Interestingly, for APPS/Intro, $n = 6$ takes the lead with +0.7%, +3.0% and +5.3%. It is very likely that the optimal window size depends on input data distribution. As for the byte level models the optimal window size is more consistent (8 bytes) across these benchmarks.

3.5. Training for multiple epochs

Multi-token training still maintains an edge on next-token prediction when trained on multiple epochs of the same data. The improvements diminish but we still have a +2.4% increase on pass@1 on MBPP and +3.2% increase on pass@100 on HumanEval, while having similar performance for the rest. As for APPS/Intro, a window size of 4 was already not optimal with 200B tokens of training.

3.6. Finetuning multi-token predictors

Pretrained models with multi-token prediction loss also outperform next-token models for use in finetunings. We evaluate this by finetuning 7B parameter models from Section 3.3